

ARCHITECTURE BRIEF

Mortgage QA/QC Engine

Compile · Run · Sources

How the compile-then-run architecture works, where the LLM is (and isn't) allowed to touch a verdict, and the pre-registered evidence behind that call.

Companion to output/THESIS.md, output/ARCHITECTURE-FOR-ARCHITECTS.md, output/RULE-COMPILATION-RESEARCH.md

1. The system in one picture

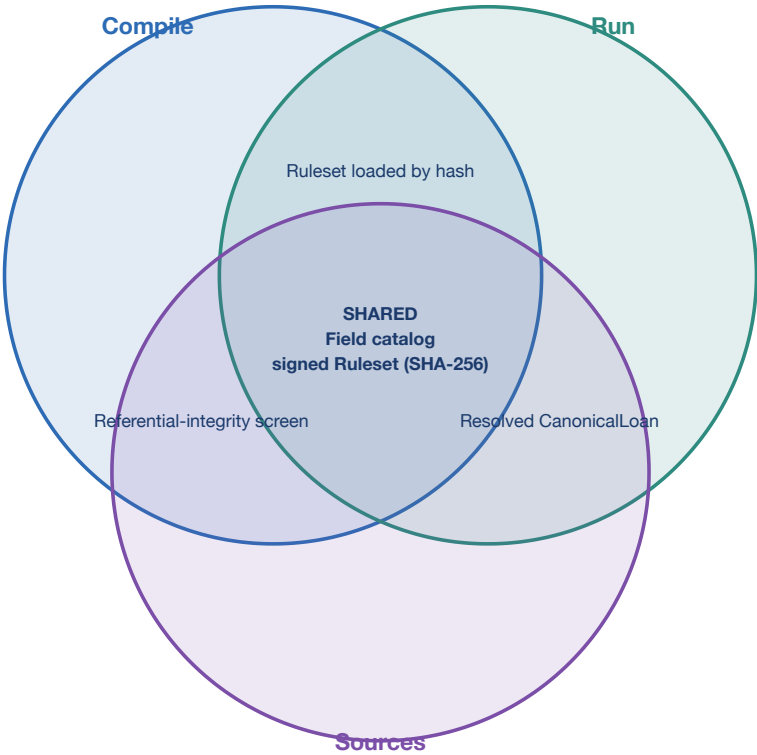
Compile turns a human-authored rule into a signed, executable check — once, reviewed, hashed. **Run** evaluates that signed check against a loan — every time, as a pure function, with no model in the loop. **Sources** keeps the closing documents (truth) and the lender's system data independent, so the comparison between them means something. All three write into one permanent audit record.



The pipeline — an LLM drafts and a human signs at compile time; nothing but arithmetic runs at evaluation time.

2. How the parts overlap

They are not three separate systems. They share one field catalog and one signed artifact, and they meet at two hard seams: the ruleset the compiler signs is exactly the ruleset the engine loads, and the loan data the engine evaluates is exactly what the source model resolved — never raw documents, never a live LLM call.



The overlap — the signed ruleset and the field catalog sit at the centre; each seam is a typed handoff, not a live call.

3. The parts side by side

Part	Purpose	Core flow	Anchor entity	Determinism guarantee
Compile	Turn lender rule text into an executable spec	AMQ row → LLM draft → screen → sign	CompiledCheckDraft	Human-reviewed; edit-distance measured
Run	Evaluate a signed ruleset against a loan	load hash → evaluate → audit record	CheckResult	Pure function; no float, no I/O, no clock

Part	Purpose	Core flow	Anchor entity	Determinism guarantee
Sources	Keep truth and system data independently comparable	doc extract → resolve system → reconcile	SourceValue (truth + sources)	Truth and system never share a code path

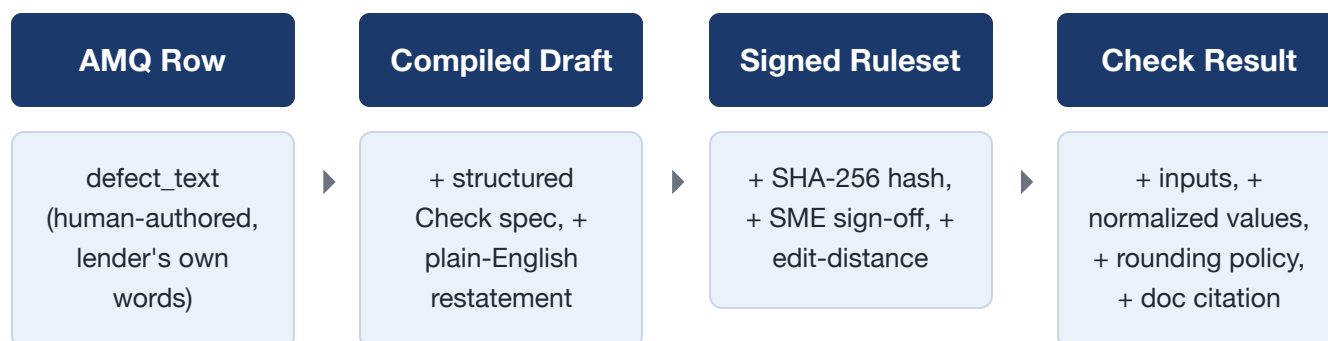
4. How the parts connect

The parts hand off typed artifacts, never live calls, across the compile/run boundary.

From	To	What flows / why
Compile	Run	The signed ruleset, loaded by SHA-256 hash — the exact rules that judged any loan are always recoverable
Sources	Run	The resolved CanonicalLoan — the engine reads pre-resolved truth/system fields, it never parses a raw document or LOS record
Compile	Sources	The referential-integrity screen — a compiled check cannot sign off referencing a field the catalog doesn't define
All three	Audit trail	RuleProvenance, RuleIntentRecord, and CheckResult all persist permanently, forming one regulator-facing record per loan

5. How the audit record grows

The artifact accumulates evidence at every stage — by the time a loan clears, the record shows the source rule text, what the LLM understood it to mean, who signed it and how much they changed, and the exact Decimal math behind the verdict.



One rule's journey — from a lender's defect-text sentence to a citable, re-derivable verdict.

6. The shared foundation

All three parts stand on the same typed vocabulary and the same math discipline — built and reviewed under one zero-regression TDD discipline, spec by spec.

Shared element	What it provides across all three
Field catalog	The one typed vocabulary Compile validates against and Run reads from — no field exists in one part's world but not another's
Decimal-only math	<code>money.py</code> — pinned <code>ROUND_HALF_EVEN</code> , fixed scale per value class, no float ever touches a money or ratio decision
Pinned engine version	<code>ENGINE_VERSION</code> is baked into every signed ruleset's hash — an engine change is itself a versioned, auditable event
Zero-regression TDD	Five specs shipped so far (001a → 003b), each spec/plan/tasks/tests before code, each merged with no regression

7. The case for compile-then-run over a runtime LLM

Each loan *could* be judged by a governed LLM at evaluation time, every run — simpler to build, and the obvious alternative. The case against that was pre-registered and tested, not assumed: a locked bake-off (`p0/experiment_g3/`) scored determinism, accuracy, and cost for the compiled engine against two runtime models, on the same six loans, before anyone saw the results.

Axis	Compiled engine	Runtime LLM — Haiku 4.5	Runtime LLM — Sonnet 4.6
Determinism (temp=0, 5 runs × 6 loans)	bit-exact	identical every run	identical every run
Accuracy (26 checks)	26/26, 0 false-clears	19/26, 1 false auto-clear	25/26, 0 false-clears
Cost @ 10,000 loans	\$0.00	≈ \$27	≈ \$70

Why compile-then-run still wins — but not for the reasons we walked in with:

- **The "LLM varies" argument is dead.** Both models were byte-identical across every run. Determinism alone doesn't distinguish the architectures at temperature=0.
- **The "\$10K/run" cost argument is dead.** Real Bedrock usage put even the expensive model at ≈\$70 per 10,000-loan run — a rounding error, not an order of magnitude.
- **What actually decided it: accuracy was model-dependent, and reproducibly wrong is worse than randomly wrong.** Haiku cleared a 98%-LTV loan against a 95% program max — a buyback-grade error — the *same way, every time*. It would pass a "run it twice" audit while still being catastrophic. Sonnet caught the same loan.
- **The compiled engine is the only path that guarantees correctness in advance**, not just after the fact. It shows the Decimal derivation an auditor can re-check by hand — a property no runtime model offers, regardless of how accurate it turns out to be.

The trade-off. A sufficiently capable runtime model can, in fact, be both reproducible and correct — this bake-off ran on six synthetic loans, decisive for proving determinism but not a population accuracy claim. If Sonnet-class accuracy holds on real, expert-labeled loans, compile-then-run becomes a *governance preference*, not a *correctness necessity* — worth re-testing honestly, not assuming away.

8. External validation — and the one gap to close

The G3 bake-off is a six-loan internal experiment. A broader literature sweep (108 agents, 25 primary sources, adversarially verified) checked whether it generalizes — and it does, on two independent benchmarks, plus surfaced one verification technique this pipeline doesn't have yet.

Source	Domain	Finding	What it confirms here
RuleArena, ACL 2025 (peer-reviewed)	Airline, tax, NBA rules	LLMs fail two ways: wrong rule picked; right rule, wrong math	Same shape as Haiku's 98%-LTV false-clear
Legal-text→DMN generation, 2026	Dutch legal decision models	Best-of-5 LLM compilation: 51–53% gold-standard match	The SME sign-off gate is load-bearing, not ceremonial
CPT-coding cost analog, AAAI 2026	Healthcare billing	Compiled engine $\approx 200\text{--}1,700\times$ cheaper than LLM-at-runtime	Independently confirms G3's cost finding, unrelated domain
Preece, Shinghal & Batarek, 1992	Rule-based systems, general	Automated anomaly detection is diagnostic, never dispositive	Validates screen-then-human-sign-off, not screen-then-auto-approve

The one gap the literature surfaced

The referential-integrity screen catches undefined field references. It cannot catch two checks that can never both pass, or a check whose guard can never fire. Formal model-checking — CTL-based conflict and unreachable detection, provably polynomial in rule count — is an established, cheap technique for exactly this class of error, and this pipeline doesn't run it yet.

Two smaller additions worth making alongside it: re-compile the same rule text and diff the resulting Check spec before sign-off — neural semantic parsing is documented to be fragile to small input changes, and this is a cheap stability test the pipeline doesn't currently run. And unify the ruleset's SHA-256 hash with its SME sign-off provenance into one tamper-evident ledger entry per version, rather than two records a reader has to correlate by hand.

9. The unifying principle

The same rule holds at every seam in this system: **push uncertainty left, into a step that is slow and reviewed; leave none in the step that runs 10,000 times a night.**

- **Compile owns the interpretation.** An LLM reads human language once, under review, and a person signs what it produced.
- **Run owns nothing but arithmetic.** No model, no network, no float — a pure function of a hash and a loan.
- **Sources own independence.** Truth and system data are typed apart so the comparison between them is never circular.
- **A person signs before anything goes live.** And that signature is measured, not assumed — an unedited ruleset is a visible red flag, not a clean pass.

The one-line takeaway

A stable wrong answer is worse than a flaky one — it survives a "show me the same number twice" audit while still buying back the loan. Compile-then-run exists to make that impossible by construction, not by hope.

10. Summary

- **The architecture pushes all interpretive uncertainty into a reviewed, signed compile step,** and leaves the evaluation step a pure, auditable function of a hash and a loan.
- **The decision to compile was tested, not assumed** — a pre-registered bake-off found two of the three original arguments false and kept the conclusion anyway, on the one axis that actually mattered: model-dependent accuracy on boundary math.
- **The recommendation holds, honestly stated:** compile-then-run is justified by auditability and guaranteed correctness, not by variance or cost — and the real-loan re-run is the test that could still change that calculus.
- **Published research independently confirms it, and points at one gap:** two 2025 benchmarks and an unrelated cost study corroborate G3's internal findings; formal model-checking for logical conflicts is the one established technique this pipeline doesn't have yet.